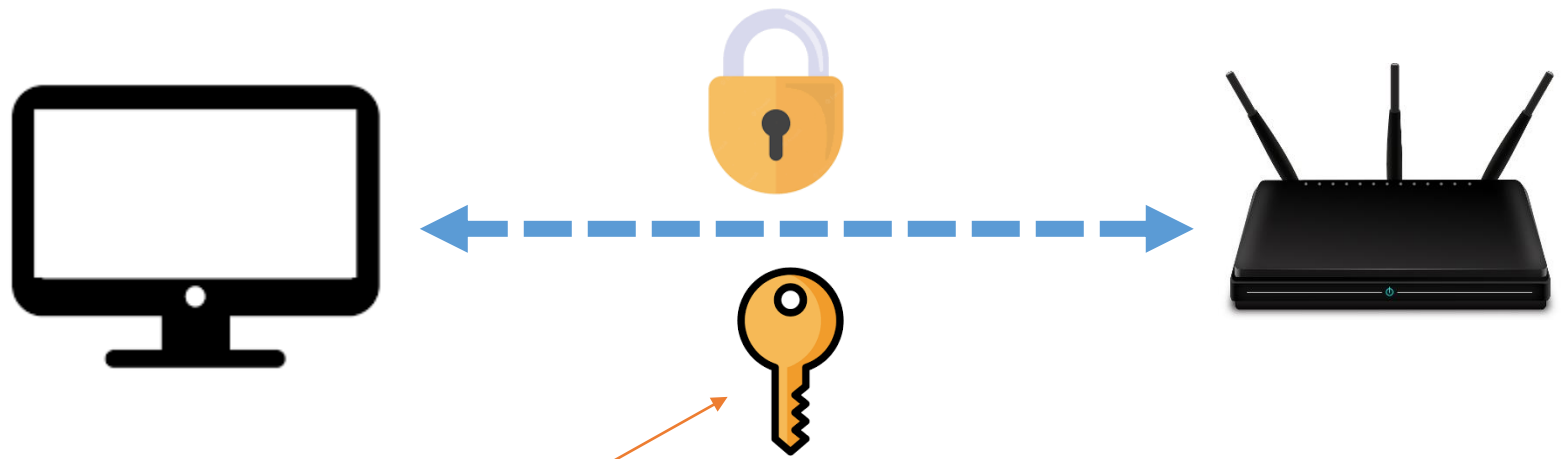




Decrypt Wireless Packets

Traffic files: https://drive.google.com/file/d/1iVHeak4ZaaZzSnUKfh_16wcjSPtWYG9L/view?usp=sharing

Overview

- Decrypt traffic
 - Find the **security key**
 - Decrypt traffics using a security key
 - wlan packets saved in .pcap
 - Extract files from packets
- Crack wifi **password**
 - IEEE 802.11 and Motivation
 - What is Wired Equivalent Privacy?



Decrypt traffics

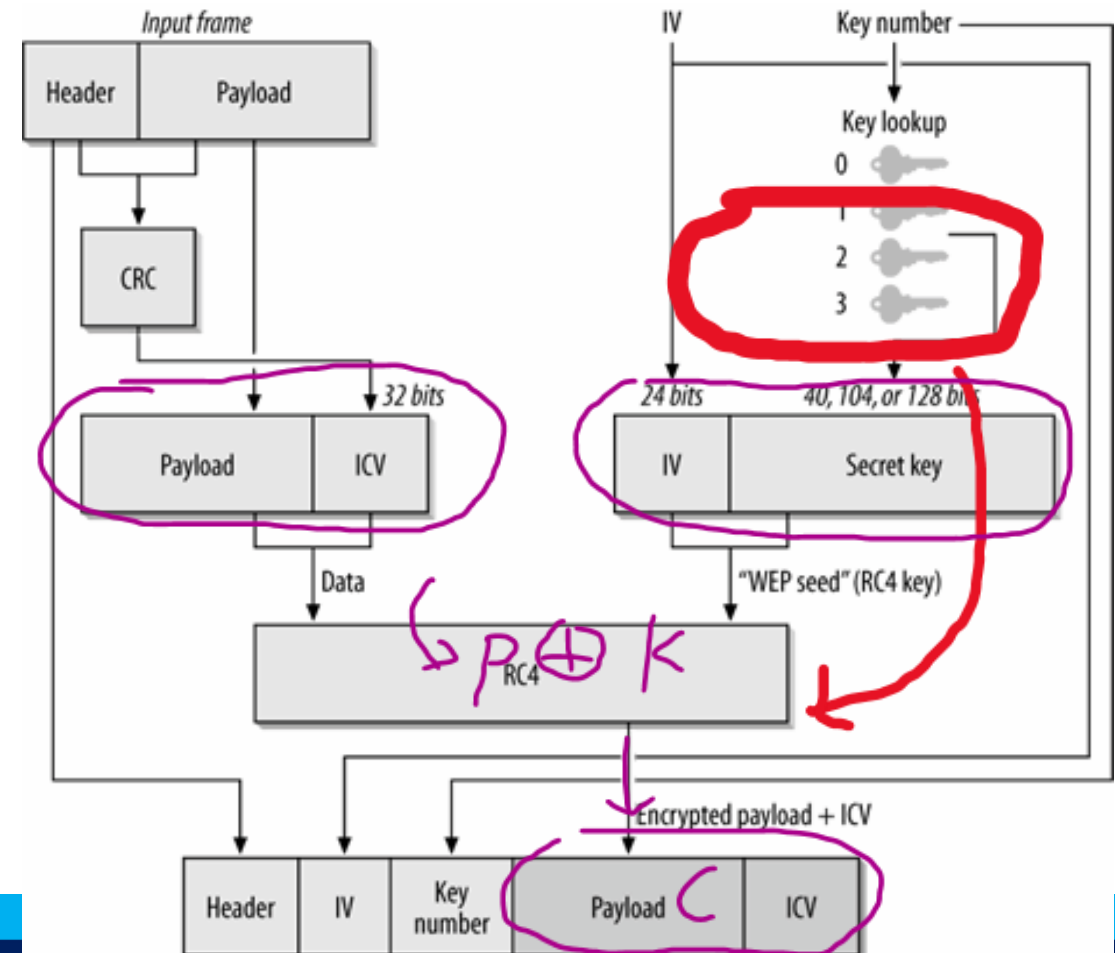
Date credits: CodeGate General Capture the flag challenge (CTF) 2015



Goal: only use a secret key (IV unknown) to decrypt encrypted payload

Problem with Stream Cipher

- Consider two packets with the same IV $\Rightarrow$ Same keystream b
- $c1 = p1 \oplus k$; $c2 = p2 \oplus k$
 - $\Rightarrow c1 \oplus c2 = p1 \oplus p2$
- two packets with the same IV
 - $\Rightarrow$ XOR = Difference in plain text
- 50% chance of using the same IV in 4823 packets.
- Recovered ICV matches $\Rightarrow$ Plain text is correct
- Possible to recover all 2^{24} keystreams in a few hours



Create a working space

```
(student@kalit01)-[~]  
$ mkdir decrypt_packets  
  
(student@kalit01)-[~]  
$ cd decrypt_packets
```

Download a traffic file

```
(student@kalit01)-[~/decrypt_packets]  
$ wget https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz  
--2023-02-09 11:10:24-- https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz  
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...  
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 12837660 (12M) [application/octet-stream]  
Saving to: 'file.xz.1'  
  
file.xz.1                100%[=====>] 12.24M  73.8MB/s   in 0.2s  
  
2023-02-09 11:10:24 (73.8 MB/s) - 'file.xz.1' saved [12837660/12837660]
```

Unzip the file

```
(student@kalit01)-[~/decrypt_packets]
$ xz -d file.xz

(student@kalit01)-[~/decrypt_packets]
$ ll
total 13904
-rw-r--r-- 1 student student 14234002 Feb  9 11:10 file
```

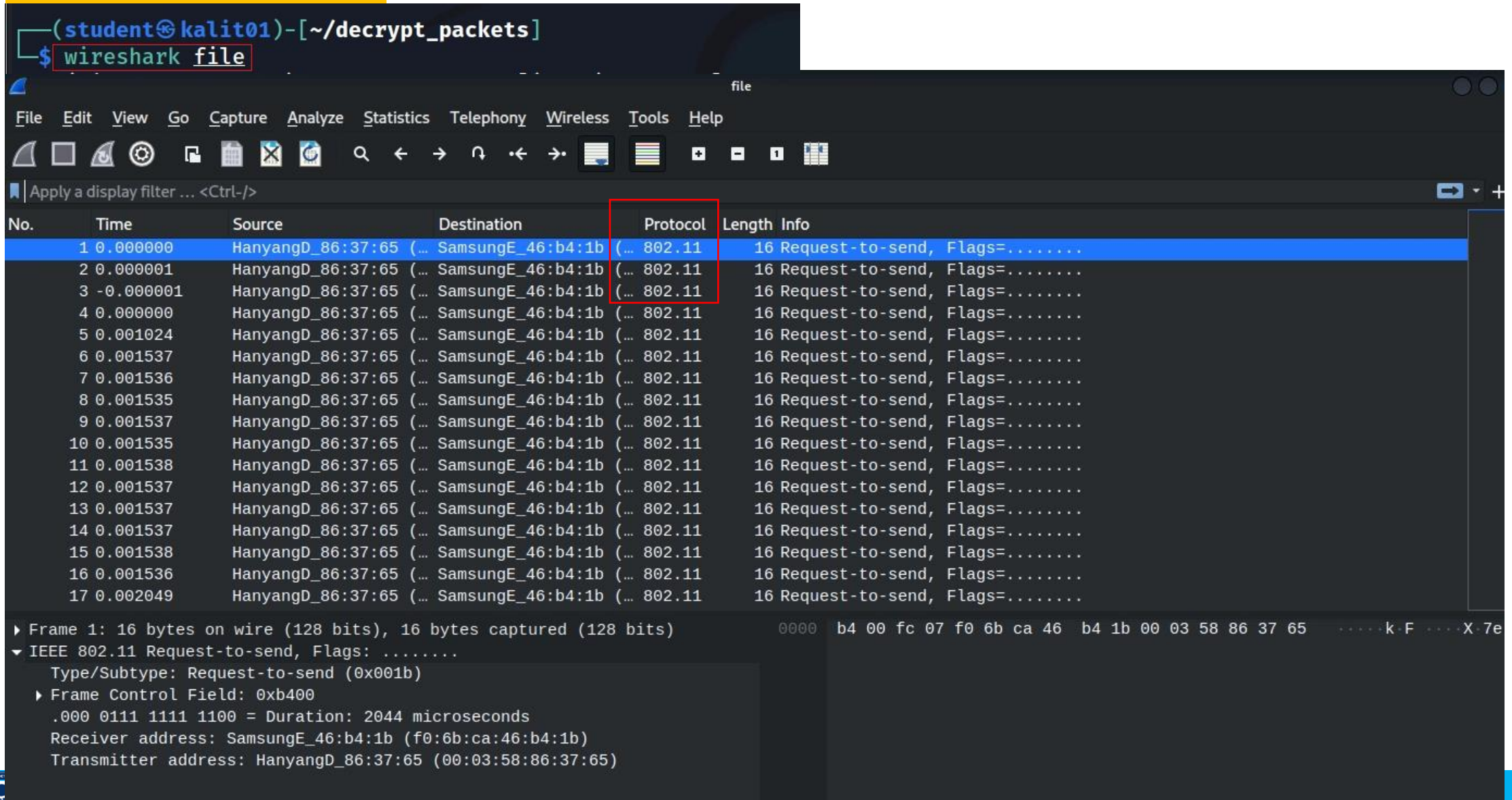
Capture the flag challenge
CodeGate General CTF 2015



Show the encrypted traffic

(student@kalit01)-[~/decrypt_packets]

\$ wireshark file



File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
2	0.000001	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
3	-0.000001	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
4	0.000000	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
5	0.001024	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
6	0.001537	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
7	0.001536	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
8	0.001535	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
9	0.001537	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
10	0.001535	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
11	0.001538	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
12	0.001537	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
13	0.001537	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
14	0.001537	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
15	0.001538	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
16	0.001536	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....
17	0.002049	HanyangD_86:37:65 (...)	SamsungE_46:b4:1b (...)	802.11	16	Request-to-send, Flags=.....

► Frame 1: 16 bytes on wire (128 bits), 16 bytes captured (128 bits) 0000 b4 00 fc 07 f0 6b ca 46 b4 1b 00 03 58 86 37 65k-F....X.7e

▼ IEEE 802.11 Request-to-send, Flags:

- Type/Subtype: Request-to-send (0x001b)
- Frame Control Field: 0xb400
 - .000 0111 1111 1100 = Duration: 2044 microseconds
 - Receiver address: SamsungE_46:b4:1b (f0:6b:ca:46:b4:1b)
 - Transmitter address: HanyangD_86:37:65 (00:03:58:86:37:65)



Find key

```
(student@kalit01)-[~/decrypt_packets]
```

```
$ aircrack-ng file
```

```
Reading packets, please wait...
```

```
Opening file
```

```
Read 45169 packets.
```

#	BSSID	ESSID	Encryption
1	00:26:66:55:97:D6	cgnetwork	WEP (15477 IVs)

```
Choosing first network as target.
```

```
Reading packets, please wait...
```

```
Opening file
```

```
Read 45169 packets.
```

```
1 potential targets
```

```
Attack will be restarted every 5000 captured ivs.
```

Aircrack-ng 1.7

[00:00:00] Tested 83 keys (got 15477 IVs)

KB	depth	byte(vote)
0	0/ 1	A4(22784) 62(20992) A8(19968) B6(19968) 42(19456) 6E(19456) 91(19200) B7(19200)
1	0/ 1	3D(23040) 51(20736) 07(20480) 62(19968) 7B(19968) 1F(19712) B0(19712) BD(19456)
2	0/ 1	F6(23808) E4(20992) D0(20736) 68(20224) 95(19712) 38(19456) 0C(19200) 45(18944)
3	1/ 10	F3(20480) C5(19968) D0(19968) 3E(19712) 43(19456) 52(19456) B2(19456) 09(19456)
4	6/ 9	01(19712) 20(19456) 3E(19456) 52(19456) 5C(19456) 90(19456) F9(19456) 45(18944)

KEY FOUND! [A4:3D:F6:F3:74]

Decrypted correctly: 100%

key length 40 bits



icdfa.edu.ng

Decrypts traffic with the key

```
(student@kalit01)-[~/decrypt_packets]
```

```
$ airdecap-ng -w A43DF6F374 file
```

```
Total number of stations seen      10
Total number of packets read      45169
Total number of WEP data packets   15477
Total number of WPA data packets    0
Number of plaintext data packets    0
Number of decrypted WEP packets    15477
Number of corrupted WEP packets     0
Number of decrypted WPA packets     0
Number of bad TKIP (WPA) packets    0
Number of bad CCMP (WPA) packets    0
```

```
(student@kalit01)-[~/decrypt_packets]
```

```
$ ll
```

```
total 26392
```

```
-rw-r--r-- 1 student student 14234002 Feb  9 11:10 file
```

```
-rw-r--r-- 1 student student 12784501 Feb  9 12:19 file-dec ←
```



```
(student@kalit01)-[~/decrypt_packets]
$ wireshark file-dec
```

The image shows a Wireshark window titled "file-dec" displaying a network traffic capture. The interface includes a menu bar (File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, Help) and a toolbar with various icons. A filter bar at the top shows "Apply a display filter ... <Ctrl-/>". The main packet list table contains 17 entries, with the first packet selected. The packet details pane at the bottom shows the structure of the first packet: Ethernet II, Src: EFMNetwo_55:97:d4 (00:26:66:55:97:d4), Dst: Broadcast (ff:ff:ff:ff:ff:ff), and Address Resolution Protocol (request).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	EFMNetwo_55:97:d4	Broadcast	ARP	42	Who has 192.168.0.15? Tell 192.168.0.1
2	0.943674	Apple_68:96:7c	NetcoreT_48:e5:b4	ARP	42	Who has 192.168.1.1? Tell 192.168.1.41
3	0.944186	Apple_68:96:7c	NetcoreT_48:a8:b5	ARP	42	Who has 192.168.0.1? Tell 192.168.0.15
4	0.944186	Apple_68:96:7c	08:10:77:92:7c:2f	ARP	42	Who has 192.168.1.1? Tell 192.168.1.12
5	0.944186	Apple_68:96:7c	EFMNetwo_55:97:d4	ARP	42	Who has 192.168.0.1? Tell 192.168.0.15
6	0.944186	Apple_68:96:7c	TendaTec_58:c4:ba	ARP	42	Who has 192.168.10.1? Tell 192.168.10.113
7	0.944185	0.0.0.0	255.255.255.255	DHCP	342	DHCP Request - Transaction ID 0x21e6be1c
8	0.944704	EFMNetwo_55:97:d4	Apple_68:96:7c	ARP	42	192.168.0.1 is at 00:26:66:55:97:d4
9	0.947778	0.0.0.0	255.255.255.255	DHCP	342	DHCP Request - Transaction ID 0x21e6be1c
10	0.949306	fe80::1c0c:f6e8:fed...	ff02::2	ICMPv6	70	Router Solicitation from f0:f6:1c:68:96:7c
11	0.954435	192.168.0.1	255.255.255.255	DHCP	590	DHCP NAK - Transaction ID 0x21e6be1c
12	0.954937	Apple_68:96:7c	Broadcast	ARP	42	ARP Announcement for 192.168.0.15
13	0.955970	fe80::1c0c:f6e8:fed...	ff02::2	ICMPv6	70	Router Solicitation from f0:f6:1c:68:96:7c
14	0.956994	Apple_68:96:7c	Broadcast	ARP	42	ARP Announcement for 192.168.0.15
15	0.959034	Apple_68:96:7c	Broadcast	ARP	42	Who has 169.254.255.255? Tell 192.168.0.15
16	0.960066	Apple_68:96:7c	Broadcast	ARP	42	Who has 169.254.255.255? Tell 192.168.0.15
17	1.026106	Apple_68:96:7c	Broadcast	ARP	42	Who has 192.168.0.12? Tell 192.168.0.15

Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0
Ethernet II, Src: EFMNetwo_55:97:d4 (00:26:66:55:97:d4), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Address Resolution Protocol (request)

0000 ff ff ff ff ff ff 00 26 66 55 97 d4 08 06 00 01& fU.....
0010 08 00 06 04 00 01 00 26 66 55 97 d4 c0 a8 00 01& fU.....
0020 00 00 00 00 00 00 c0 a8 00 0f& fU.....



Extract files from packets



"Foremost" is a popular open-source forensic tool used for data recovery on the Linux operating system

```
(student@kalit01) - [~/decrypt_packets]
$ foremost -v file-dec
Foremost version 1.5.7 by Jesse Kornblum, Kris Kendall, and Nick Mikus
Audit File

Foremost started at Thu Feb  9 16:10:04 2023
Invocation: foremost -v file-dec
Output directory: /home/student/decrypt_packets/output
Configuration file: /etc/foremost.conf
Processing: file-dec
|-----
File: file-dec
Start: Thu Feb  9 16:10:04 2023
Length: 12 MB (12784501 bytes)

Num      Name (bs=512)      Size      File Offset      Comment
0:      00000104.jpg      258 KB      53549
1:      00004538.jpg       87 KB      2323670
2:      00000406.gif       35 B       208185      (1 x 1)
3:      00001260.gif       35 B       645564      (1 x 1)
4:      00002450.gif       42 B      1254588      (1 x 1)
5:      00003653.gif       35 B      1870635      (1 x 1)
6:      00003719.gif       35 B      1904213      (1 x 1)
7:      00010456.gif       49 B      5353479      (1 x 1)
8:      00010876.gif       35 B      5568641      (1 x 1)
9:      00001639.htm       67 B       839396
10:     00024427.htm        1 KB      12506741
11:     00001765.png       283 B       904100      (44 x 44)
12:     00001831.png       166 B       937711      (12 x 12)
13:     00010214.png        67 B      5229704      (1 x 1)
14:     00010671.png       110 B      5464006      (1 x 400)
15:     00020736.png       425 B      10616910      (19 x 19)
16:     00024242.png       589 B      12412171      (19 x 19)
*|
Finish: Thu Feb  9 16:10:04 2023

17 FILES EXTRACTED

jpg:= 2
gif:= 7
htm:= 2
png:= 6
```

Show extracted files

```
(student@kalit01)-[~/decrypt_packets]  
$ ll  
total 26396  
-rw-r--r-- 1 student student 14234002 Feb  9 11:10 file  
-rw-r--r-- 1 student student 12784501 Feb  9 12:19 file-dec  
drwxr-xr-- 6 student student    4096 Feb  9 16:10 output
```

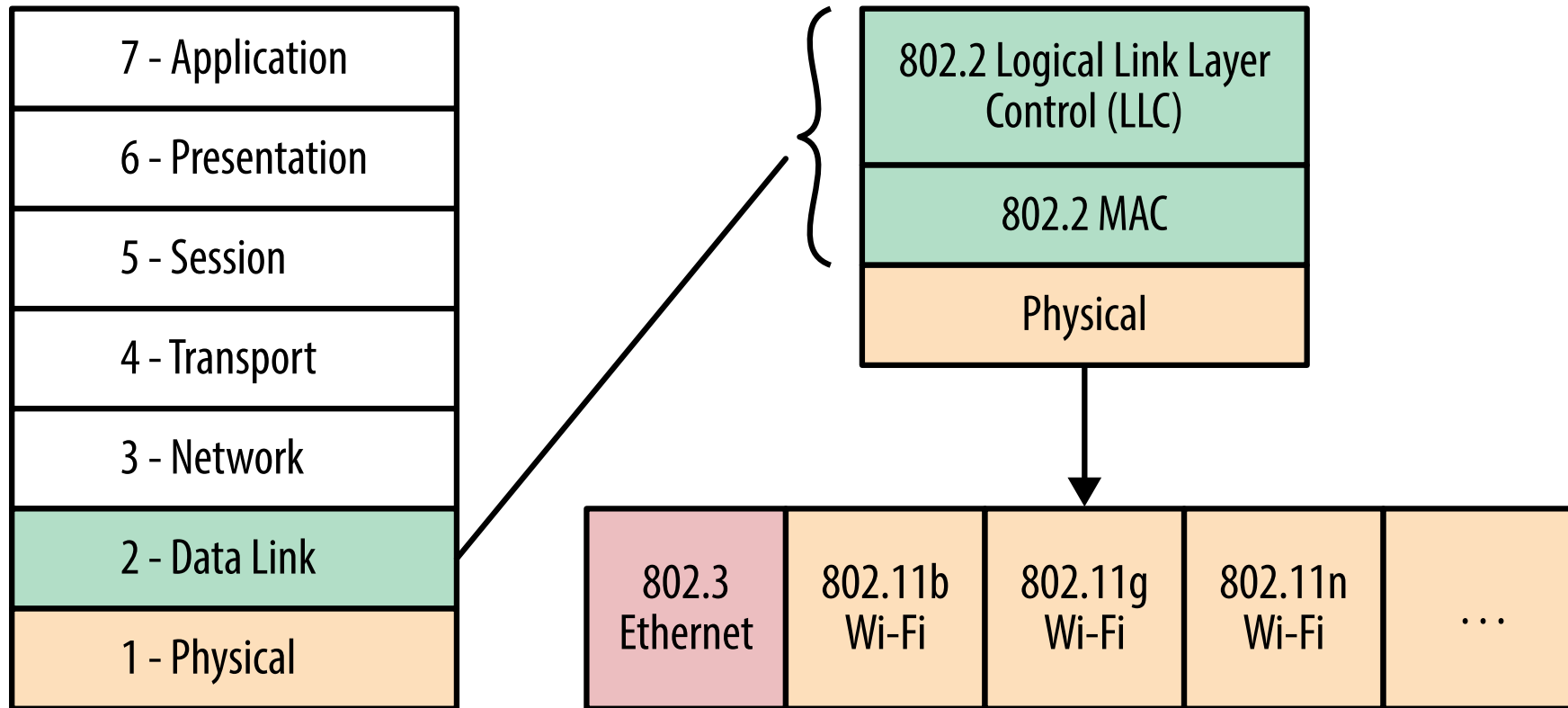
```
(student@kalit01)-[~/decrypt_packets]  
$ ls output  
audit.txt  gif  htm  jpg  png
```



Crack wifi password



Where does 802.11 fit in the OSI Model?



(Ethernet) and 802.11 (WiFi) data and physical layers

Header	Payload	Trailer(ICV)
--------	---------	--------------

IEEE 802.11

- **ICV**: Integrity Check Value = cyclic redundancy check (CRC) value
- $ICV = CRC(Header + Payload)$

- 802.11 is a set of standards for wireless networking that defines how wireless devices communicate with each other.
- The 802.11 standards define the format of data frames, which are the units of data that are transmitted between devices.
- There are several types of 802.11 frames, including management frames, control frames, and data frames.
 - Management frames: manage the wireless network, such as establishing and maintaining a connection, authenticating devices, and to configure the network.
 - Control frames: control the flow of data on the network, such as requesting or acknowledging the transmission of data.
 - Data frames: transmit data between devices on the network.

IEEE 802.11 Supports Encryption Protocols

- Encryption Protocols
 - Wired Equivalent Privacy (WEP),
 - Wi-Fi Protected Access (WPA), and
 - Wi-Fi Protected Access II (WPA2).
- These protocols use various methods to encrypt the data transmitted in 802.11 frames
 - symmetric key encryption,
 - public key encryption, and
 - message authentication codes.
- Users can choose to use encryption or not
 - depends on the configuration of the network and the security needs of its users.

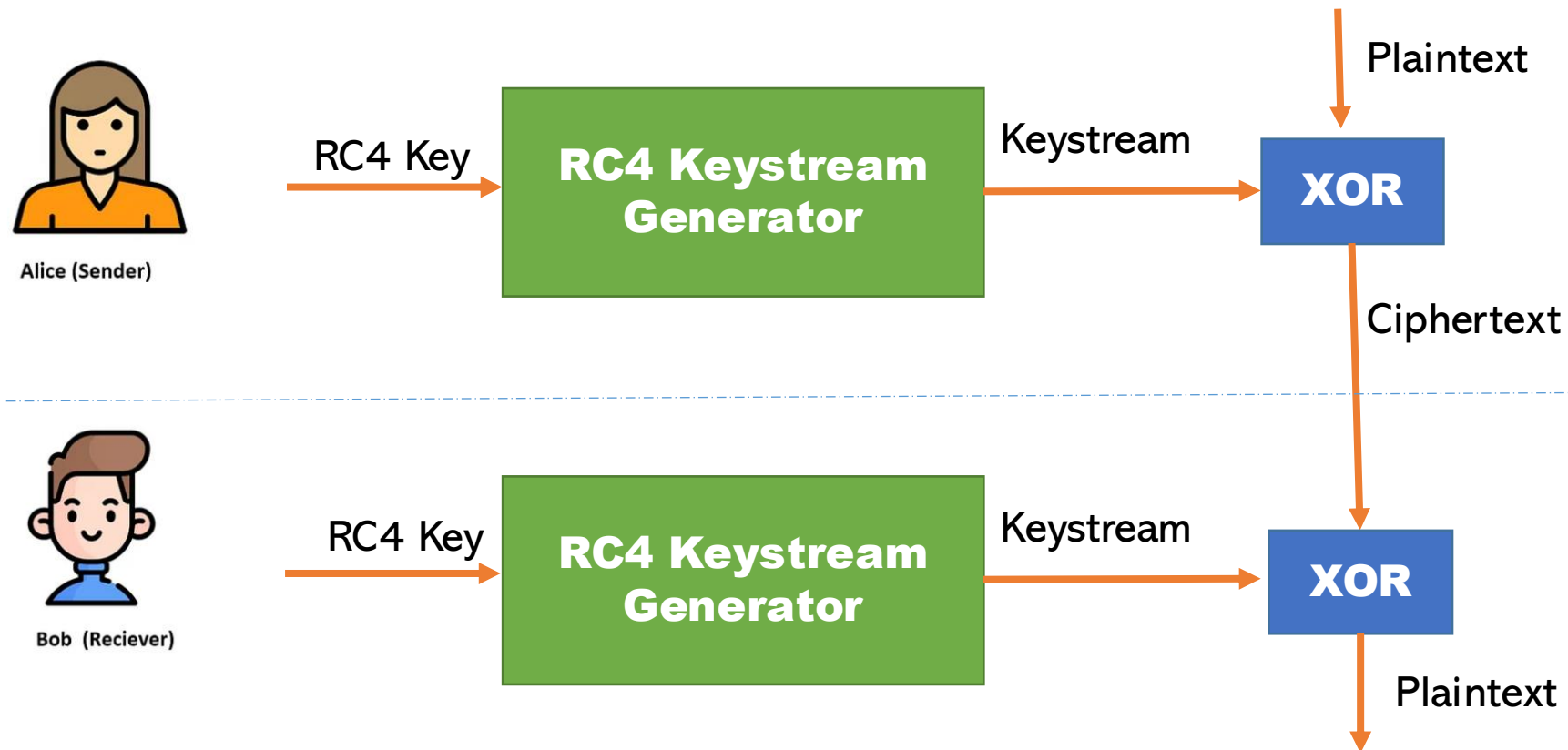


Wired Equivalent Privacy (WEP)

- It is an outdated security protocol used to protect wireless local area networks (WLANs).
- It was one of the first security standards developed for wireless networks and was intended to provide a level of security comparable to that of a wired network.
- WEP is stream cipher using RC4 to generate a keystream
- WEP key length -64/128/256 bits
 - 128-bit WEP remains one of the most common implementations



RC4 Algorithm



RC4 Keystream Generator

- Initialization/key scheduling
 - The key is used to initialize a 256-byte state array (S) with the values 0 to 255.
 - The state array is then shuffled based on the key.
- Keystream generation
 - At each step, the generator uses two index variables, i and j , to generate a keystream byte.
 - The i - th and j - th elements of the state array are swapped, and
 - the sum of i and the j - th element is used to determine the next keystream byte.
 - This process is repeated until enough keystream bytes have been generated to encrypt the plaintext.



# Harry Sauers	
# rc4.py	
# demo of RC4 encryption algorithm	
def key_scheduling(key):	def stream_generation(sched):
sched = [i for i in range(0, 256)]	stream = []
	i = 0
i = 0	j = 0
for j in range(0, 256):	while True:
i = (i + sched[j] + key[j % len(key)]) % 256	i = (1 + i) % 256
	j = (sched[i] + j) % 256
tmp = sched[j]	
sched[j] = sched[i]	tmp = sched[j]
sched[i] = tmp	sched[j] = sched[i]
	sched[i] = tmp
return sched	yield sched[(sched[i] + sched[j]) % 256]



x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0

XOR stream cipher

Unencrypted
802.11 data frame

1010	1100	...	...	1110	0101
------	------	-----	-----	------	------



keystream

1110	0100	...	...	1000	1001
------	------	-----	-----	------	------

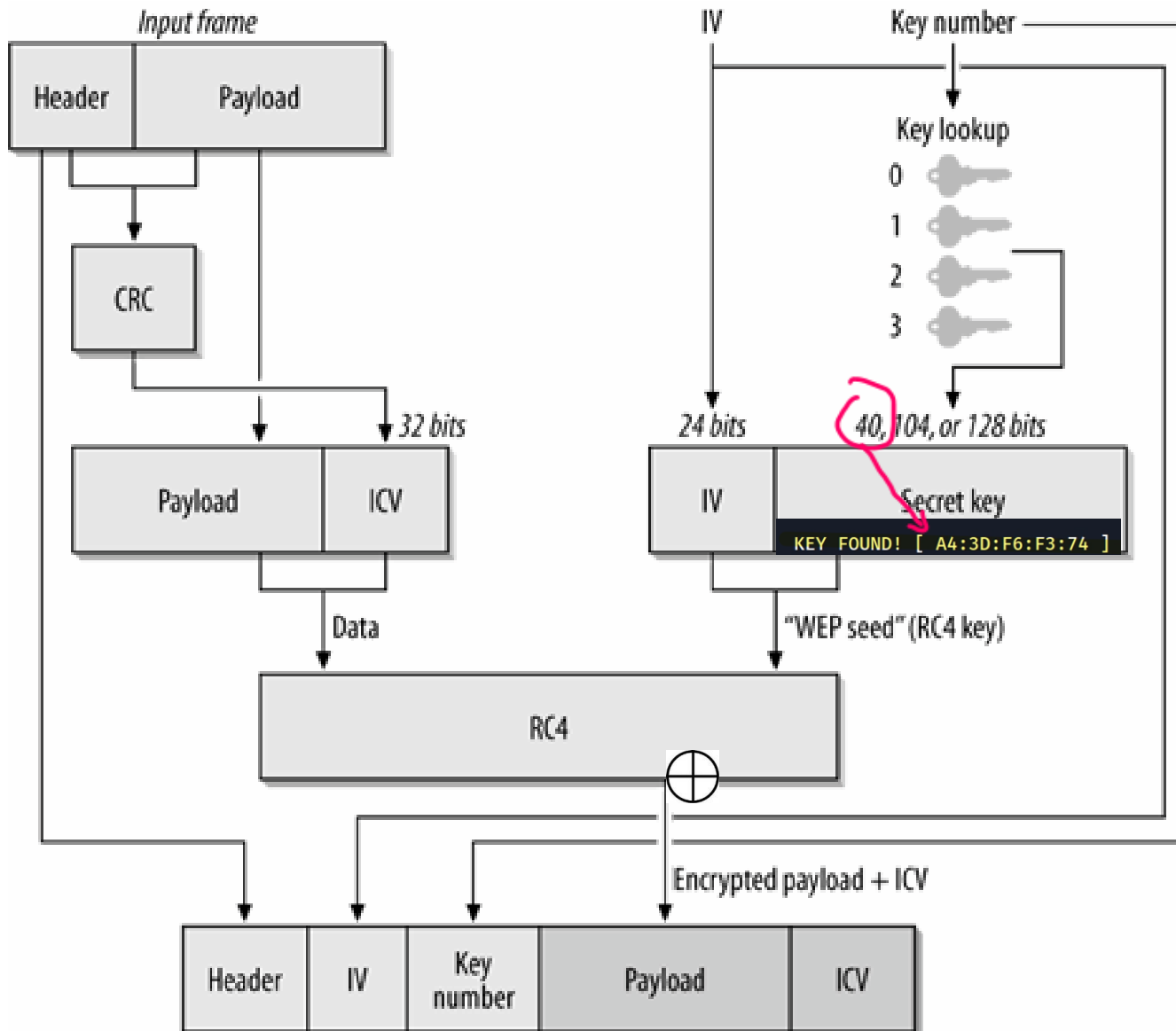
RC4

=

802.11 WEP data
frame

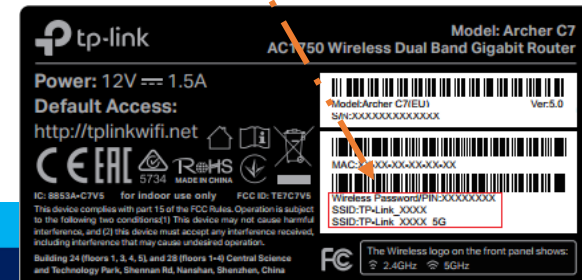
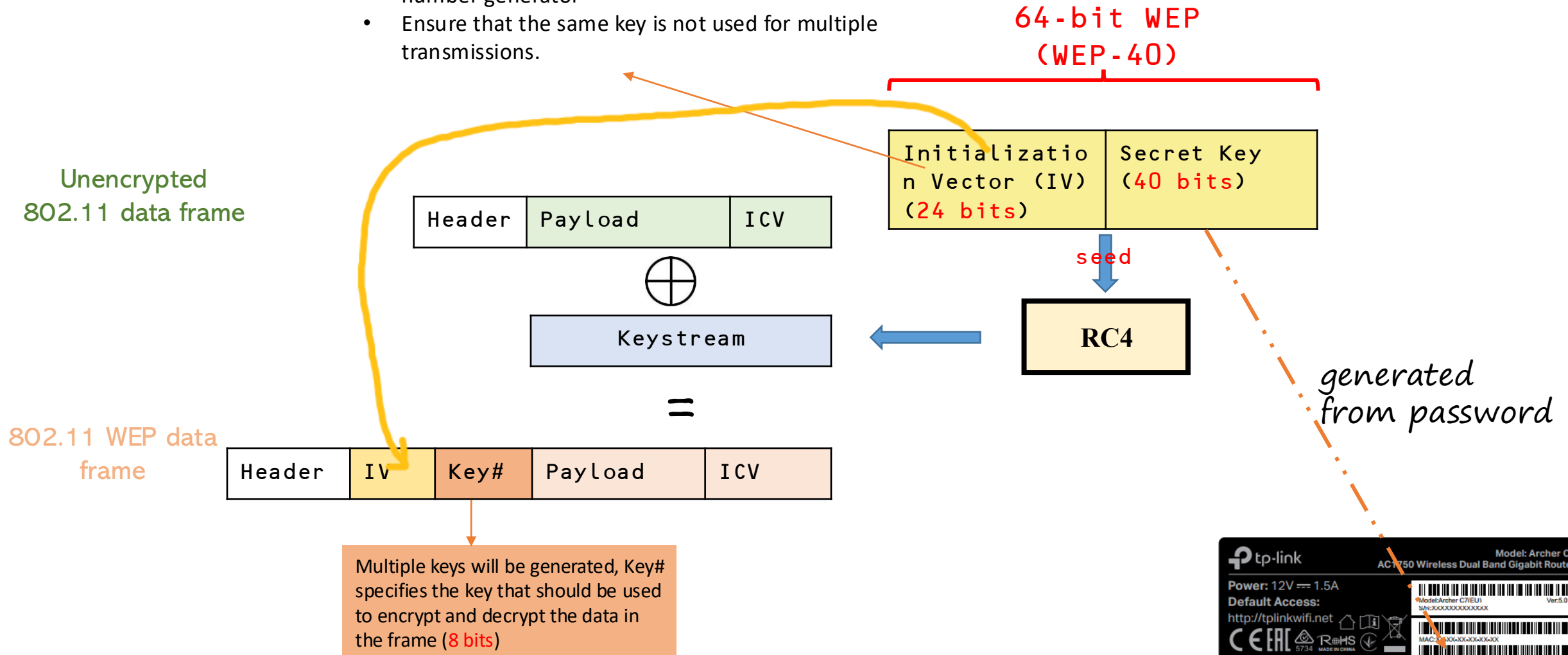
0100	1000	...	...	0110	1100
------	------	-----	-----	------	------





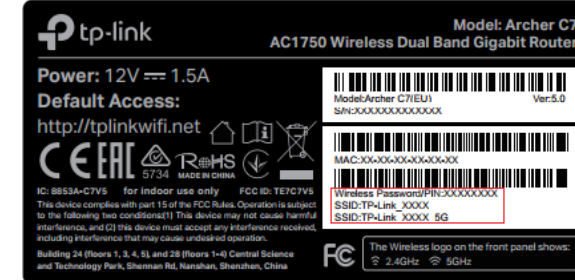
- WEP also allows for multiple keys to be stored and used at the same time.
 - Specifically, WEP allows for four different keys to be stored simultaneously, and the user can switch between the keys at any time.
- This allows for greater flexibility in managing the security of a wireless network.

- Generated by a cryptographically secure random number generator
- Ensure that the same key is not used for multiple transmissions.



Secret Key Management

- Secret keys are generated with a passphrase/password
 - one passphrase generates multiple keys
- Multiple keys provide additional security
 - if an attacker were to capture and crack one key, other keys would still be in use
- Keys are manually distributed
- Every device on the network uses one key to encrypt every transmitted packet
 - an eavesdropper could intercept the WEP encryption and find the key.



5 ASCII characters
(0-9, a-z, A-Z)

128-bit WEP
WEP 104: 13 ASCII
characters (0-9, a-
z, A-Z)

A screenshot of a web-based configuration interface for a router, specifically the WEP security settings. The 'Security Mode' is set to 'WEP'. Under 'Default Transmit Key', key 1 is selected. 'WEP Encryption' is set to '128 bits 26 hex digits'. A 'Passphrase' of 'mypassword' is entered, with a 'Generate' button next to it. Below, four keys are displayed: Key 1: 5214A76CF00DE3DBE0CC89D287, Key 2: 5B4385F918D62BFEC6BA3DB65E, Key 3: F660A66F24042390A033808522, and Key 4: 4449DFB55C3E709BCD98D20CD9.

Select a WEP Password

- To create a 64/40 WEP Hex Password, you need to enter a combination of 10 digits. You can choose any letter from A-F or any number from 0-9. Sample HEX WEP Password: 0FB310FF28.
- To create a 64/40 WEP ASCII, you need to enter a combination of 5 ASCII characters. Sample ASCII WEP Password: hello.
- To create a 128/104 WEP Hex Password, you need to enter a combination of 26 digits. You can choose any letter from A-F or any number from 0-9. Sample HEX WEP Password: 0FB310FF280FB310FF28123456.
- To create a 128/104 WEP ASCII, you need to enter a combination of 13 ASCII characters. Sample ASCII WEP Password: hellohello123.

Password Tips:

Use a mix of letters and numbers. Don't use personal information that could be guessed or easily discovered (for example, names of family members, birthdates, phone numbers)

2.4 GHz Wi-Fi

Select a WEP Password:

64/40 bit	✓
HEX	✓

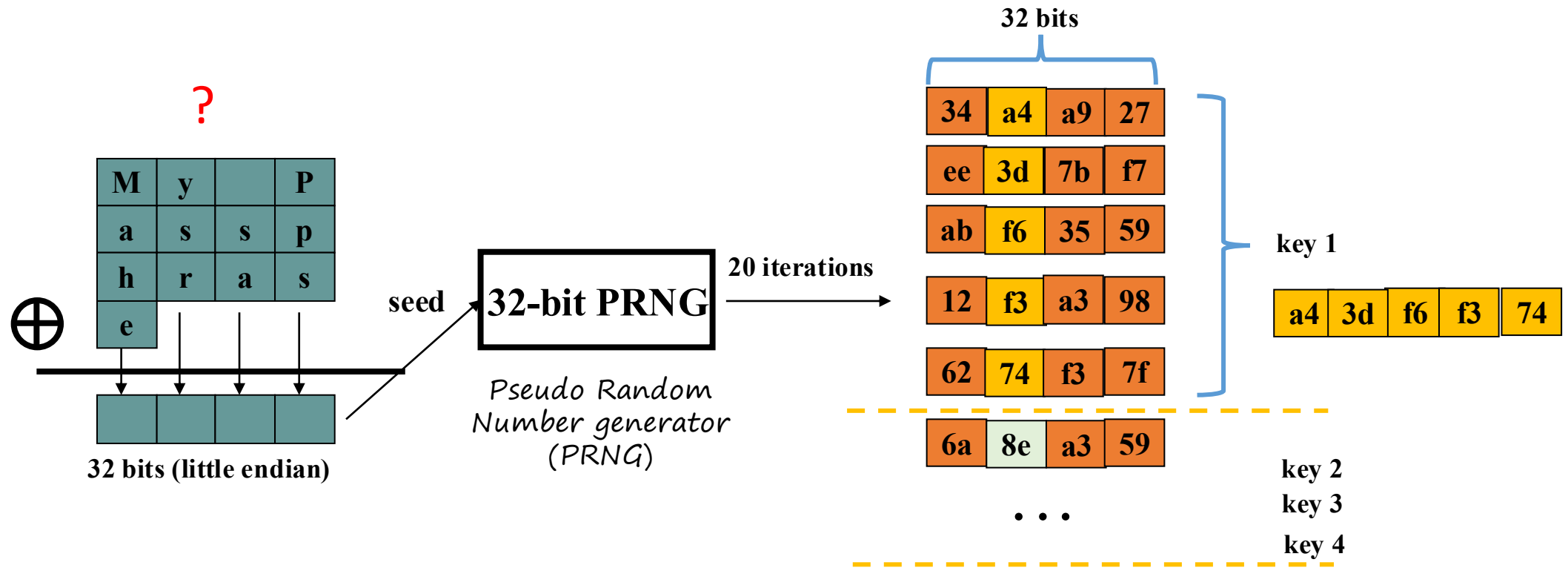
2.4 GHz Wi-Fi Password:

0123456789

0 Digits Left



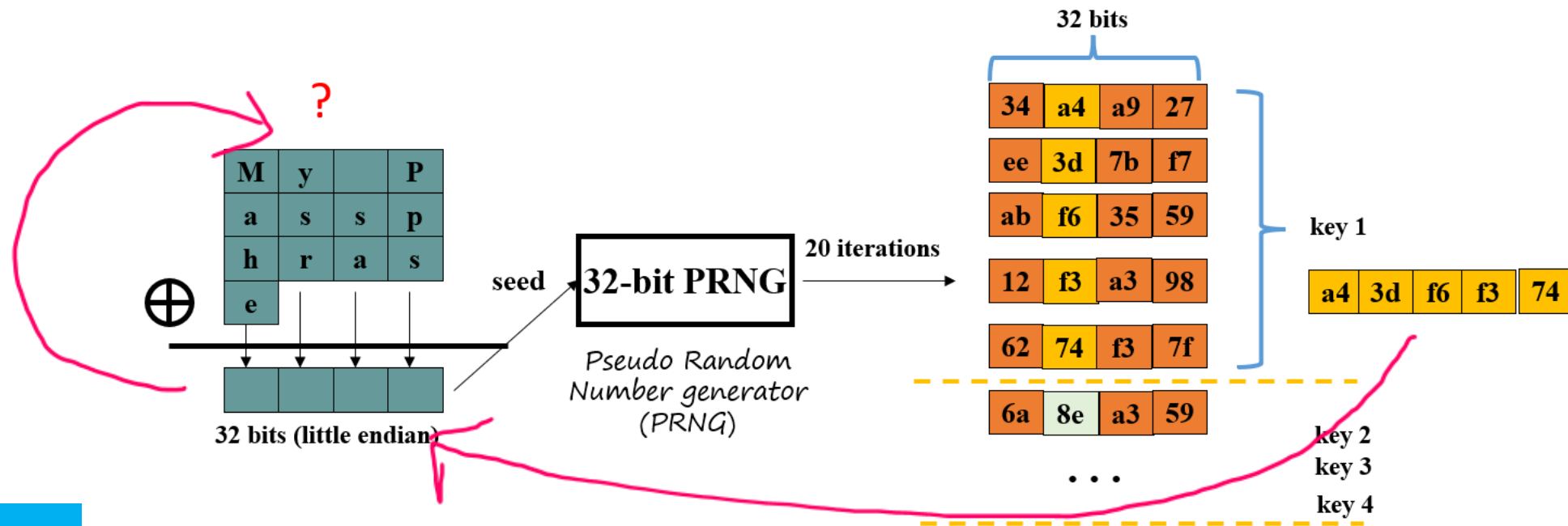
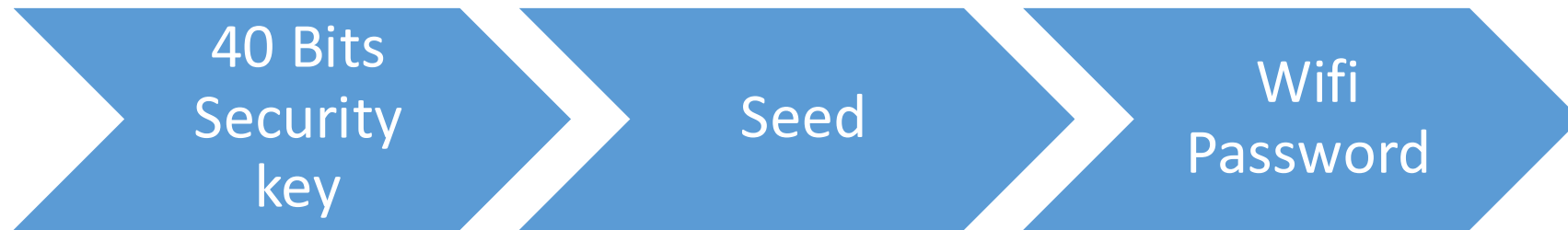
How Are WEP-40 Secret Keys Generated?



Seeding: ASCII string mapped to 32-bits

PRNG: Generate 32-bits key candidates

How to find password



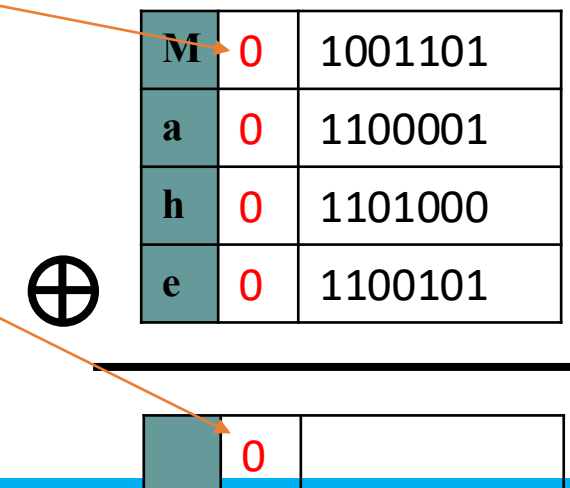
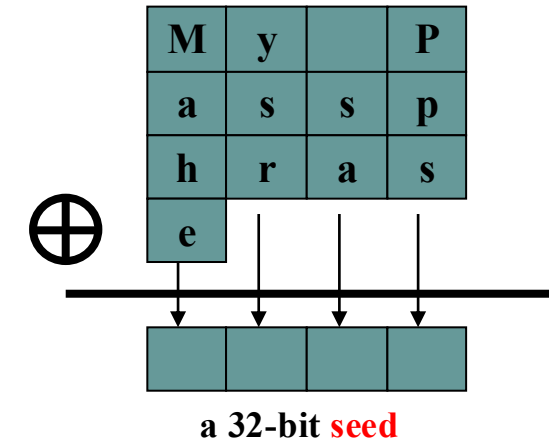
WEP-40 Secret Keys Generator has a security problem

- WEP-40 security key should have 40-bits of entropy
- Key entropy is reduced in several ways
 - seed entropy reduced
 - PRNG entropy reduced



Seeding problem – reduced seed entropy (32-28 bits)

- XOR operation guarantees produce four zero bits
 - Input is ASCII. High bit of each character is always zero
- XOR of these high bits is also zero
 - Only seeds 00:00:00:00 through 7f:7f:7f:7f can occur (entropy is reduced to 28 bits = 32-4)



PRNG problem: Reduced PRNG Entropy (32-24 bits)

- PRNG is a Linear congruential generator

The generator is defined by the recurrence relation:

$$X_{n+1} = (aX_n + c) \bmod m$$

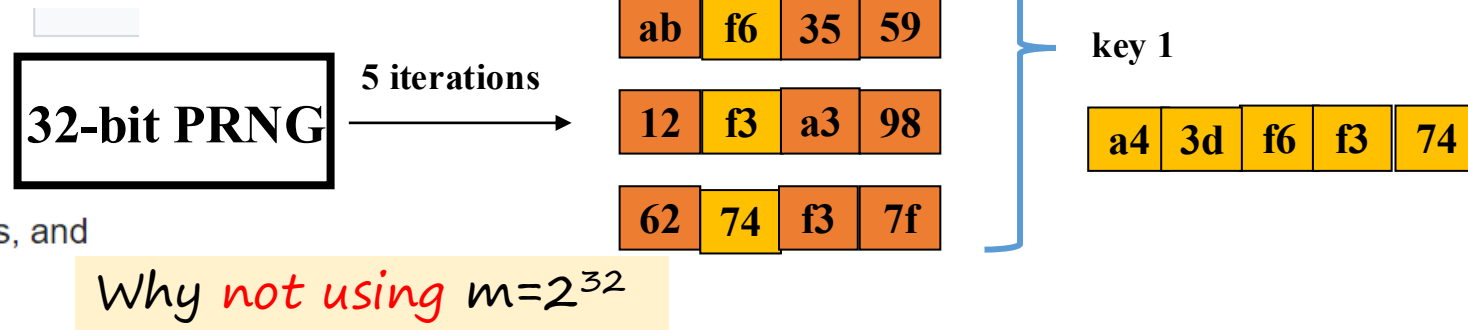
where X is the sequence of pseudo-random values, and

m , $0 < m$ — the "modulus" $m=2^{24}$

a , $0 < a < m$ — the "multiplier"

c , $0 \leq c < m$ — the "increment"

X_0 , $0 \leq X_0 < m$ — the "seed" or "start value"



- Because the most significant byte of the outputs is not used, which is equivalent to generating only 24-bits words at each iteration.
- Only seeds 00:00:00:00 through 00:ff:ff:ff (24 bits) may result in unique keys!



Find a seed

$$X_{n+1} = (aX_n + c) \bmod m$$

x_0 =seed (We want to figure it out)

$a=0x000343fd$ (how do we know, next slide)

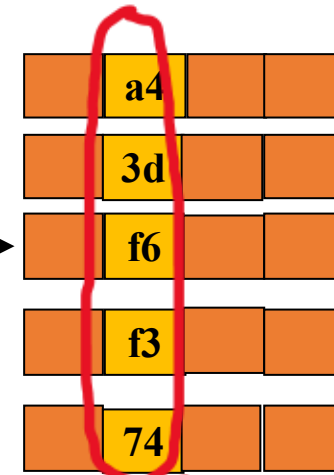
$c=0x269ec3$ (how do we know)

$m=0x00FFFFFF$ (2^{24})

?

32-bit PRNG

5 iterations



we know

Approach: Generate 5 candidate keys, x_1, x_2, x_3, x_4, x_5 , their second most significant bytes must match the secret keys. The x_1 is the seed

Parameters in common use [\[edit \]](#)

The following table lists the parameters of LCGs in common use, including built-in *rand()* functions in [runtime libraries](#) of various [compilers](#). This table is to show popularity, not examples to emulate; *many of these parameters are poor*. Tables of good parameters are available.^{[10][2]}

Source	modulus <i>m</i>	multiplier <i>a</i>	increment <i>c</i>	output bits of seed in <i>rand()</i> or <i>Random(L)</i>
ZX81	$2^{16} + 1$	75	74	
Numerical Recipes from the "quick and dirty generators" list, Chapter 7.1, Eq. 7.1.6 parameters from Knuth and H. W. Lewis	2^{32}	1664525	1013904223	
Borland C/C++	2^{32}	22695477	1	bits 30..16 in <i>rand()</i> , 30..0 in <i>lrand()</i>
glibc (used by GCC) ^[17]	2^{31}	1103515245	12345	bits 30..0
ANSI C : Watcom , Digital Mars , CodeWarrior , IBM VisualAge C/C++ ^[18] C90 , C99 , C11 : Suggestion in the ISO/IEC 9899 , ^[19] C17	2^{31}	1103515245	12345	bits 30..16
Borland Delphi , Virtual Pascal	2^{32}	134775813	1	bits 63..32 of (<i>seed</i> × <i>L</i>)
Turbo Pascal	2^{32}	134775813 (8088405 ₁₆)	1	
Microsoft Visual/Quick C/C++	2^{32}	214013 (343FD ₁₆)	2531011 (269EC3 ₁₆)	bits 30..16



```

1 # https://www.vnsecurity.net/ctf%20-%20clgt%20crew/
2 # 2015/03/16/codegate-good-crypto.html
3
4 key=[0xa4,0x3d,0xf6,0xf3,0x74]
5 a, c, m = 0x000343fd, 0x269ec3, 0xFFFFFFFF
6 secret_key_bytes=5
7
8 for seed in range(1<<24):
9     correct = True
10    x = seed
11
12    for i in range(secret_key_bytes):
13        x = (a * x + c) & m
14        # shift 16 bits to right
15        # the same as check the second most significant byte
16        if (x>>16)!=key[i]:
17            correct = False
18            break
19
20    if correct:
21        print("Found it: ", hex(seed))
22        print("Verify: ")
23
24        for i in range(secret_key_bytes):
25            seed = (a* seed + c) & m
26            print(hex(seed>>16),hex(seed))

```

only need 24

Found it: 0x12766b

Verify:

0xa4 0xa4a782

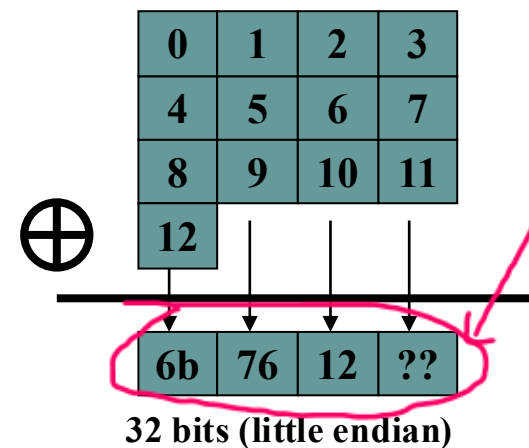
0x3d 0x3d303d

0xf6 0xf6420c

0xf3 0xf3089f

0x74 0x74c0e6

Password Position



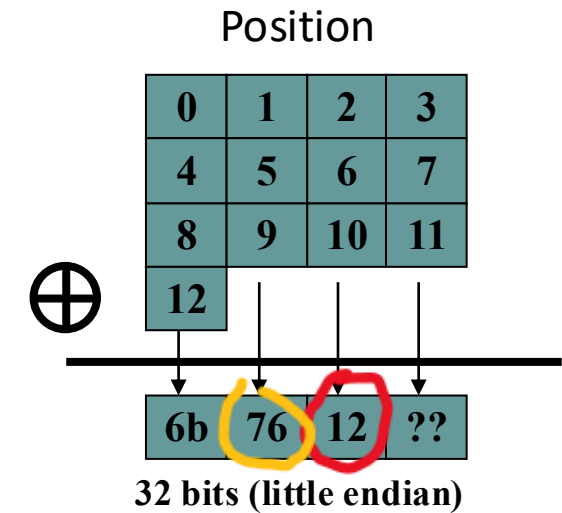
We know the password = [A-Za-z]+

32 bits seed

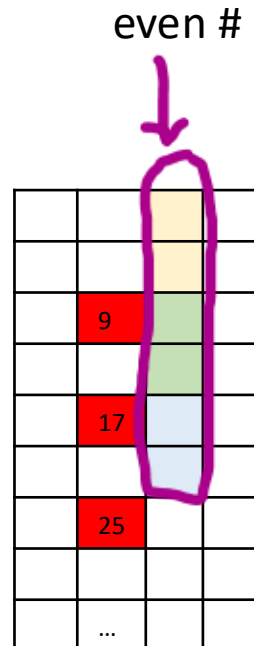


How long is the passphrase?

Character	Decimal Number	Binary Number	Character	Decimal Number	Binary Number
blank space	32	0010 0000	^	94	0101 1110
!	33	0010 0001	-	95	0101 1111
"	34	0010 0010	`	96	0110 0000
#	35	0010 0011	a	97	0110 0001
\$	36	0010 0100	b	98	0110 0010
A	65	0100 0001	c	99	0110 0011
B	66	0100 0010	d	100	0110 0100
C	67	0100 0011	e	101	0110 0101
D	68	0100 0100	f	102	0110 0110
E	69	0100 0101	g	103	0110 0111
F	70	0100 0110	h	104	0110 1000
G	71	0100 0111	i	105	0110 1001
H	72	0100 1000	j	106	0110 1010
I	73	0100 1001	k	107	0110 1011
J	74	0100 1010	l	108	0110 1100
K	75	0100 1011	m	109	0110 1101
L	76	0100 1100	n	110	0110 1110
M	77	0100 1101	o	111	0110 1111
N	78	0100 1110	p	112	0111 0000
O	79	0100 1111	q	113	0111 0001
P	80	0101 0000	r	114	0111 0010
Q	81	0101 0001	s	115	0111 0011
R	82	0101 0010	t	116	0111 0100
S	83	0101 0011	u	117	0111 0101
T	84	0101 0100	v	118	0111 0110
U	85	0101 0101	w	119	0111 0111
V	86	0101 0110	x	120	0111 1000
W	87	0101 0111	y	121	0111 1001
X	88	0101 1000	z	122	0111 1010
Y	89	0101 1001	{	123	0111 1011
Z	90	0101 1010		124	0111 1100
[	91	0101 1011	}	125	0111 1101
/	92	0101 1100	~	126	0111 1110
]	93	0101 1101			



- $0x12 = 001,0010$ can only be generated by an even number of [A-Za-z] characters. Why?
- If generated by an odd number of [A-Aa-z], 0 should be 1.
- Possible passphrase char ends at (include) 9, 17, 25, etc,
- How about $0x76 =$

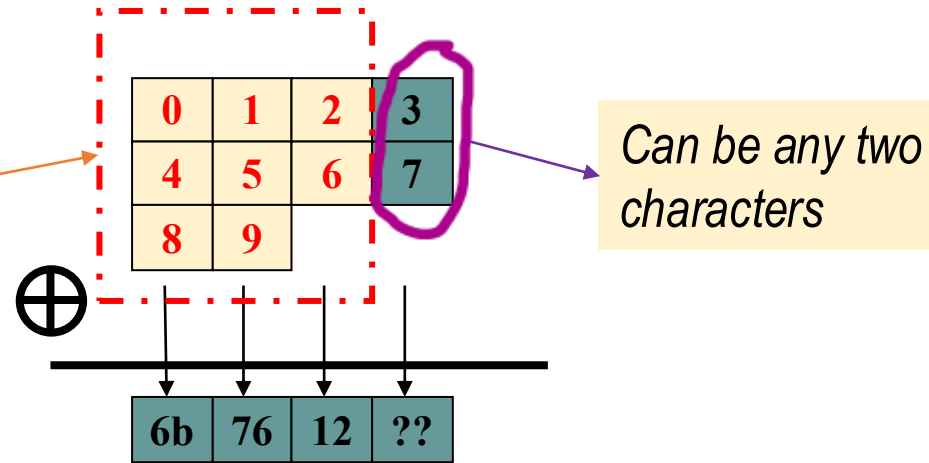


0111,0101



The length of the passphrase is 10

- We only need to brute-force **8** characters.
 - All possibility combinations = 52^8



The hash of the 10-character needs to match the hint provided in the game **0xff7b948953ac**.

Information for cracking passphrase

- We know the 40-bit secret key.

a4	3d	f6	f3	74
----	----	----	----	----

 - It can be computed by capturing and analyzing WEP packets
 - We will demonstrate it later
- We know the seed of the secret key

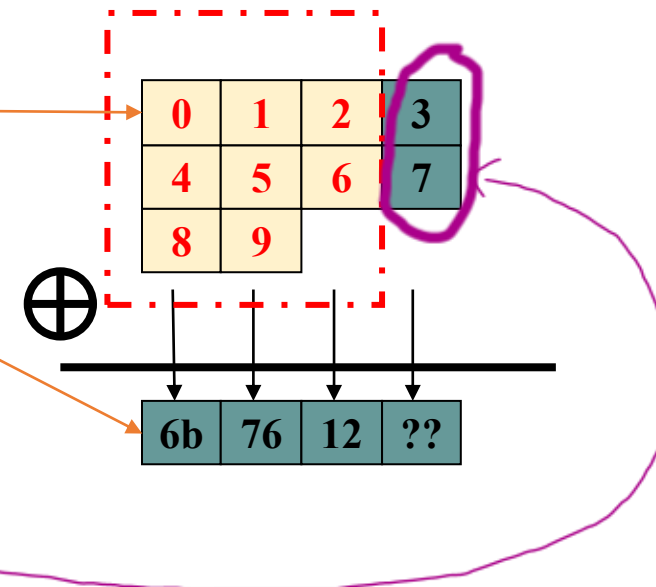
6b	76	12	??
----	----	----	----
- The passphrase consists of all lowercase
 - Analyzing the seed
- We know the hash code of the passphrase **0xff7b948953ac**
 - it is provided by CTF



```

1 import itertools
2 import string
3 import hashlib
4
5
6 def generate_combinations():
7     characters = string.ascii_lowercase
8     combinations = itertools.product(characters, repeat=8)
9     # For testing only
10    combinations = ["aaaaaaaa", "cgwpkexz", "cgwpkexa", "cgwpkexa", "cgwpkexa", "cgwpkexa"]
11    return combinations
12
13
14 def check_xor_sum(combination):
15     if ord(combination[0]) ^ ord(
16         combination[3]) ^ ord(combination[6]) != 0x6b :
17         return False
18     if ord(combination[1]) ^ ord(
19         combination[4]) ^ ord(combination[7]) != 0x76:
20         return False
21     if ord(combination[2]) ^ ord(combination[5]) != 0x12:
22         return False
23     return True
24
25
26 def insert_letters(valid_combination):
27     new_combinations = []
28     characters = 'abcdefghijklmnopqrstuvwxyz'
29     for x in characters:
30         for y in characters:
31             new_combination = tuple(list(valid_combination[:3]) + [x] +
32                                     list(valid_combination[3:6]) + [y] +
33                                     list(valid_combination[6:]))
34             new_combinations.append(new_combination)
35     return new_combinations
36
37 def check_shal_hash(combination):
38     combination_str = ''.join(combination)
39     shal_hash = hashlib.shal(combination_str.encode('utf-8'))
40     return shal_hash.hexdigest().startswith('ff7b948953ac')
41
42
43 cracked= False
44 for combination in generate_combinations():
45     # print("process: ", ''.join(combination))
46     if check_xor_sum(combination):
47         print("Match xor sum: ", ''.join(combination))
48         for comb_x_y in insert_letters(combination):
49             if check_shal_hash(comb_x_y):
50                 cracked = True
51                 print("Cracked: ", ''.join(comb_x_y))
52                 break
53     if cracked:
54         break
55
56

```



- https://drive.google.com/file/d/1iVHeak4ZaaZzSnUKfh_16wcjSPtWYG9L/view?usp=sharing
- python
crackWEP40_noGPU_Improved_v1.py

```

student@seed-labs01: ~/m
student@seed-labs01: ~/myfo
Match xor sum: cgwobegs
Match xor sum: cgwocegr
Match xor sum: cgwodegu
Match xor sum: cgwoeegt
Match xor sum: cgwofegw
Match xor sum: cgwogegV
Match xor sum: cgwohegy
Match xor sum: cgwoiegx
Match xor sum: cgwokegz
Match xor sum: cgwopega
Match xor sum: cgworegc
Match xor sum: cgwosegb
Match xor sum: cgwotege
Match xor sum: cgwouegd
Match xor sum: cgwovegg
Match xor sum: cgwowegf
Match xor sum: cgwoxegi
Match xor sum: cgwoyegh
Match xor sum: cgwozegk
Match xor sum: cgwpaexp
Match xor sum: cgwpbexs
Match xor sum: cgwpcexr
Match xor sum: cgwpdexu
Match xor sum: cgwpeext
Match xor sum: cgwpfexw
Match xor sum: cgwpgexv
Match xor sum: cgwphexy
Match xor sum: cgwpiexx
Match xor sum: cgwpkexz
Cracked: cgwepkeyxz

```



Assignment

- What are the IP addresses of the host and server?
- What are the Mac addresses?
- Can you manually extract pictures and HTML pages?
- What is the password of the router?



To do

It seems Wireshark can crack 40-bit secret key



Decrypt using Wireshark is not working, not sure why

